

XP Robotics — Multi-AGV Simulation Project Report

1. Project Brief

This project delivers a high-precision, multi-robot autonomous navigation simulation built in **ROS 2 Humble** and **Gazebo**. The environment successfully operates three distinct Automated Guided Vehicles (AGVs) simultaneously, each running a different localization and control architecture for comparative analysis.

The primary achievement of this project is the optimization of the Nav2 stack for **Robot 3**, which consistently achieves **sub-1.5 cm arrival accuracy** at a transit speed of **0.6 m/s** across all tested waypoints.

2. Standard Operating Procedure (SOP)

Follow these steps to launch the full simulation, run tests, and monitor telemetry in real time.

Step 1: Clean Environment & Build Workspace

Always ensure the ROS 2 environment is clean before launching, to prevent shared memory locks.

```
killall -9 gzserver gzclient rviz2 python3 ros2
rm -rf /dev/shm/*
cd ~/Documents/XP_Robotics/StepFile_gazebo/AGV/agv_simulation_workspace
colcon build
source install/setup.bash
```

Step 2: Launch the Simulation

Open **Terminal 1**, source the workspace, and launch to spawn the environment, the 3 AGVs, and their respective localization/navigation stacks.

```
cd ~/Documents/XP_Robotics/StepFile_gazebo/AGV/agv_simulation_workspace
source install/setup.bash
ros2 launch agv_multi_robot_navigation.launch.py
```

(Wait until Gazebo and RViz are fully loaded and the map is visible before proceeding.)

Step 3: Launch the Telemetry Dashboard

Open **Terminal 2** to start the Flask web server for real-time visualization.

```
cd ~/Documents/XP_Robotics/StepFile_gazebo/AGV/telemetry_and_graphs/web_da
python3 app.py
```

Access the dashboard in your web browser at: <http://127.0.0.1:5000>

Step 4: Execute Robot Testing

Open **Terminal 3**, source the workspace, and run the desired orchestrator to command the robots. To test Robot 3 specifically:

```
cd ~/Documents/XP_Robotics/StepFile_gazebo/AGV/agv_simulation_workspace
source install/setup.bash
ros2 run agv test_orchestrator
```

Target Robot	Command
Robot 1	ros2 run agv random_orchestrator --ros-args -p robot_name:=robot1
Robot 2	ros2 run agv random_orchestrator --ros-args -p robot_name:=robot2
Robot 3	ros2 run agv test_orchestrator

(Each command above must be run after cd-ing into the workspace and sourcing install/setup.bash, as shown for Robot 3.)

3. Robot Control Structures & Technologies

Each robot leverages a distinct architecture to enable direct comparative analysis of localization and control strategies.

Robot 1 — EKF Localization + Custom PID

- **Technology:** robot_localization (Extended Kalman Filter). Operates blind (no static map), fusing wheel odometry and IMU data.
- **Control Strategy:** Custom Python **PID controller** that mathematically calculates heading and distance error to drive the cmd_vel outputs.

Robot 2 — AMCL Localization + Custom PID

- **Technology:** Nav2 amcl (Adaptive Monte Carlo Localization). Localizes against a pre-generated 2D occupancy grid map using LiDAR scans.
- **Control Strategy:** Custom Python **PID controller**, relying on the accurate AMCL map-to-odom transform for position feedback rather than raw EKF drift.

Robot 3 — AMCL + Full Nav2 Stack

- **Technology:** Complete **ROS 2 Navigation Stack (Nav2)** using `bt_navigator` (Behavior Trees).
- **Control Strategy:** `RegulatedPurePursuitController`.
- **Tuning Highlights:**
 - Goal tolerance: **0.013 m** (1.3 cm)
 - Approach velocity for micro-adjustments: **0.005 m/s**
 - Max transit velocity: **0.6 m/s**

Robot Localization		Control Strategy	Map-Dependent
Robot 1	EKF (<code>robot_localization</code>)	Custom PID	No
Robot 2	AMCL (LiDAR)	Custom PID	Yes
Robot 3	AMCL (LiDAR)	Nav2 Full Stack (Behavior Trees + Regulated Pure Pursuit)	Yes

4. Final Accuracy Validation

Robot 3 was autonomously tested against 6 distinct waypoints across the map.

Performance Output:

Waypoint (x, y) Minimum Arrival Error

-3.5, 3.5	1.00 cm
3.5, 3.5	1.41 cm
3.5, -3.5	1.41 cm
-3.5, -3.5	1.00 cm
0.0, 0.0	1.00 cm
-2.0, 2.0	1.00 cm

- **Average arrival error:** 1.14 cm
- **Best-case accuracy:** 1.00 cm
- **Worst-case accuracy:** 1.41 cm

Conclusion: The simulation reliably demonstrates that Robot 3 can operate autonomously at 0.6 m/s and reach target coordinates with an absolute precision of **1.00 to 1.41 cm**, staying within the configured 1.3 cm goal tolerance in the majority of trials.